Assignment 4: Satellite Attitude Kinematics and Dynamics

1 Introduction

In this assignment the student will learn how to simulate the attitude kinematics and dynamics of a satellite. This will be done by extending simutils.py and sat_lib.py with functionality for simulating satellite attitude kinematics and dynamics

After completing this assignment the student will learn:

- How to implement and simulate the attitude kinematics and dynamics of rigid-bodies.
- How to implement and simulate the attitude kinematics and dynamics specifically for satellites.

2 Preliminaries

Before describing the steps needed to complete this assignment, some preliminaries are needed. Specifically, coordinate transformations, attitude representations, attitude kinematics, rigid-body attitude dynamics. This section is mainly based on Lecture 5 and 6.

2.1 Notation

The set of real numbers is denoted $\mathbb{R}$, and an n-dimensional real vector space can be constructed as the cartesian product of n set of real numbers, i.e. $\mathbb{R}^n \equiv \mathbb{R}\times\cdots\times\mathbb{R}$. We denote reference frames by $\mathcal{F}^{(\cdot)}$, for instance a reference frame A, is written as $\mathcal{F}^A$. Vectors are written using lowercase bold symbols for instance $\boldsymbol{u}$, while matrices use upper case for instance $\boldsymbol{A}$. The derivative with respect to time is denoted by $\dot{}$, for instance $\frac{d\boldsymbol{u}}{dt} = \dot{\boldsymbol{u}}$. Superscripts are used to indicate the reference frame the vector's coordinates is expressed in, such that the vector $\boldsymbol{u}^A$ has coordinates expressed in $\mathcal{F}^A$[(i)]. In other words a vector $\boldsymbol{u}^A$ can be written as a column matrix $\mathbb{R}^{3\times 1}$[(ii)] using its coordinates x, y, z in $\mathcal{F}^A$ as

$$\boldsymbol{u}^A = \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} x & y & z \end{bmatrix}^\top$$

where $^\top$ denotes transpose. A coordinate transformation between two frames $\mathcal{F}^A$ and $\mathcal{F}^B$ is denoted $\boldsymbol{R}^A_B$, such that $\boldsymbol{u}^A = \boldsymbol{R}^A_B\boldsymbol{u}^B$. Angular velocities are denoted with both sub and superscripts as $\boldsymbol{\omega}^C_{AB} \in \mathbb{R}^3$ which is the angular velocity of $\mathcal{F}^B$ relative to $\mathcal{F}^A$, with coordinates expressed in $\mathcal{F}^C$. By the derivative of a vector we mean the rate of change of its coordinates, for instance for a vector $\boldsymbol{u}^A(t) = \begin{bmatrix} x(t) & y(t) & z(t) \end{bmatrix}^\top$

$$\frac{d\boldsymbol{u}^A}{dt} = \lim_{h\to 0} \frac{\boldsymbol{u}^A(t+h) - \boldsymbol{u}^A(t)}{h} = \begin{bmatrix} \lim_{h\to 0} \frac{x(t+h)-x(t)}{h} \\ \lim_{h\to 0} \frac{y(t+h)-y(t)}{h} \\ \lim_{h\to 0} \frac{z(t+h)-z(t)}{h} \end{bmatrix} = \dot{\boldsymbol{u}}^A(t)$$

2.2 Rigid-Bodies

As we know a rigid-body can be thought of as a collection of N particles with $3N$ degrees of freedom. Since it is assumed that the distance between any two particles remains constant there are $3N - 6$ constraints. Therefore only 6 independent coordinates are needed to fully specify its configuration/pose (position and orientation). To be able to assign these coordinates we make use of two reference frames

$\mathcal{F}^i$**:** An intertial frame with origin O_i,

$\mathcal{F}^b$**:** A body frame rigidly attached to the rigid-body (non-inertial) with origin O_b located at its center of mass.

(i) Typically we just say $\boldsymbol{u}^A$ is expressed in $\mathcal{F}^A$

(ii) Usually just written as $\mathbb{R}^3$

Then first three independent coordinates are the position of its center of mass[iii], which is the average of the particle positions weighted with their mass

$$\boldsymbol{p}^i = \frac{\sum_{i=0}^{N-1} m_i \boldsymbol{p}_i^i}{\sum_{i=0}^{N-1} m_i} = \frac{\sum_{i=0}^{N-1} m_i \boldsymbol{p}_i^i}{m}$$

where m is its total mass. Its orientation represented as a coordinate transformation between the two reference frames specifies the last three independent coordinates.

2.3 Reference Frames and coordinate transformations

Consider two reference frames $\mathcal{F}^A$ with unit vector $\boldsymbol{i}$, $\boldsymbol{j}$ and $\boldsymbol{k}$ and $\mathcal{F}^B$ with unit vectors $\boldsymbol{I}$, $\boldsymbol{J}$ and $\boldsymbol{K}$. If we have a vector $\boldsymbol{u}^A$ it can be written as a linear combination of the unit vectors of $\mathcal{F}^A$

$$\boldsymbol{u}^A = u_x \boldsymbol{i} + u_y \boldsymbol{j} + u_z \boldsymbol{k}$$

since $\boldsymbol{i}$, $\boldsymbol{j}$ and $\boldsymbol{k}$ are also vectors they can be expressed as a linear combination of $\boldsymbol{I}$, $\boldsymbol{J}$ and $\boldsymbol{K}$

$$\begin{aligned}
\boldsymbol{i} &= i_x \boldsymbol{I} + i_y \boldsymbol{J} + i_z \boldsymbol{K} \\
\boldsymbol{j} &= j_x \boldsymbol{I} + j_y \boldsymbol{J} + j_z \boldsymbol{K} \\
\boldsymbol{k} &= k_x \boldsymbol{I} + k_y \boldsymbol{J} + k_z \boldsymbol{K}
\end{aligned}$$

if we substitute for $\boldsymbol{i}$, $\boldsymbol{j}$, $\boldsymbol{k}$ it will give us $\boldsymbol{u}^B$[iv]

$$\boldsymbol{u}^B = (u_x i_x + u_y j_x + u_z k_x)\,\boldsymbol{I} + (u_x i_y + u_y j_y + u_z k_y)\,\boldsymbol{J} + (u_x i_z + u_y j_z + u_z k_z)\,\boldsymbol{K}$$

which can be written as a matrix vector product

$$\boldsymbol{u}^B = \begin{bmatrix} u_x i_x + u_y j_x + u_z k_x \\ u_x i_y + u_y j_y + u_z k_y \\ u_x i_z + u_y j_z + u_z k_z \end{bmatrix} = \begin{bmatrix} i_x & j_x & k_x \\ i_y & j_y & k_y \\ i_z & j_z & k_z \end{bmatrix} \begin{bmatrix} u_x \\ u_y \\ u_x \end{bmatrix} = \boldsymbol{R}_A^B \boldsymbol{u}^A$$

The matrix $\boldsymbol{R}_A^B$ represent the coordinate transformation from vectors with coordinates in $\mathcal{F}^A$ to the same vector but with coordinates in $\mathcal{F}^B$[v] and can be witten as

$$\boldsymbol{R}_A^B = \begin{bmatrix} \boldsymbol{i} \bullet \boldsymbol{I} & \boldsymbol{j} \bullet \boldsymbol{I} & \boldsymbol{k} \bullet \boldsymbol{I} \\ \boldsymbol{i} \bullet \boldsymbol{J} & \boldsymbol{j} \bullet \boldsymbol{J} & \boldsymbol{k} \bullet \boldsymbol{J} \\ \boldsymbol{i} \bullet \boldsymbol{K} & \boldsymbol{j} \bullet \boldsymbol{K} & \boldsymbol{k} \bullet \boldsymbol{K} \end{bmatrix}$$

which are just the projections of $\boldsymbol{i}$, $\boldsymbol{j}$ and $\boldsymbol{k}$ onto $\boldsymbol{I}$, $\boldsymbol{J}$ and $\boldsymbol{K}$ and using the definition of the scalar/dot product is equivalent to

$$\boldsymbol{R}_A^B = \begin{bmatrix} \cos\theta_{iI} & \cos\theta_{jI} & \cos\theta_{kI} \\ \cos\theta_{iJ} & \cos\theta_{jJ} & \cos\theta_{kJ} \\ \cos\theta_{iK} & \cos\theta_{jK} & \cos\theta_{kK} \end{bmatrix}$$

[iii] Or equivalently the location of O_b with respect to O_i, $\boldsymbol{p}^i = \overrightarrow{O_i O_b}$

[iv] We would like to emphasize that $\boldsymbol{u}^A$ and $\boldsymbol{u}^B$ are the same vectors in that they have the same magnitude and direction. They are just written as a linear combination of different sets of unit vectors.

[v] In other words a coordinate transformation preserves the length and direction of a vector.

where $\cos\theta_{(\cdot)}$ is called a direction cosine. The coordinate transformation $\boldsymbol{R}_B^A$ can be found by repeating the same steps but from $\mathcal{F}^B$ to $\mathcal{F}^A$[vi] to yield[vii]

$$\boldsymbol{R}_B^A = \begin{bmatrix} I_x & J_x & K_x \\ I_y & J_y & K_y \\ I_z & J_z & K_z \end{bmatrix} = \begin{bmatrix} \boldsymbol{I} \bullet \boldsymbol{i} & \boldsymbol{J} \bullet \boldsymbol{i} & \boldsymbol{K} \bullet \boldsymbol{i} \\ \boldsymbol{I} \bullet \boldsymbol{j} & \boldsymbol{J} \bullet \boldsymbol{j} & \boldsymbol{K} \bullet \boldsymbol{j} \\ \boldsymbol{I} \bullet \boldsymbol{k} & \boldsymbol{J} \bullet \boldsymbol{k} & \boldsymbol{K} \bullet \boldsymbol{k} \end{bmatrix}$$

knowing that $\boldsymbol{J} \bullet \boldsymbol{i} = \boldsymbol{i} \bullet \boldsymbol{J}$ such that

$$\boldsymbol{R}_B^A = \begin{bmatrix} \boldsymbol{i} \bullet \boldsymbol{I} & \boldsymbol{i} \bullet \boldsymbol{J} & \boldsymbol{i} \bullet \boldsymbol{K} \\ \boldsymbol{j} \bullet \boldsymbol{I} & \boldsymbol{j} \bullet \boldsymbol{J} & \boldsymbol{j} \bullet \boldsymbol{K} \\ \boldsymbol{k} \bullet \boldsymbol{I} & \boldsymbol{k} \bullet \boldsymbol{J} & \boldsymbol{k} \bullet \boldsymbol{K} \end{bmatrix} = \begin{bmatrix} \cos\theta_{iI} & \cos\theta_{iJ} & \cos\theta_{iK} \\ \cos\theta_{jI} & \cos\theta_{jJ} & \cos\theta_{jK} \\ \cos\theta_{kI} & \cos\theta_{kJ} & \cos\theta_{kK} \end{bmatrix}$$

which is just the transpose of $\boldsymbol{R}_A^B$. Therefore since

$$\boldsymbol{u}^B = \boldsymbol{R}_A^B \boldsymbol{u}^A = \boldsymbol{R}_A^B \boldsymbol{R}_B^A \boldsymbol{u}^B = \boldsymbol{R}_A^B \left(\boldsymbol{R}_A^B\right)^\top \boldsymbol{u}^B = \boldsymbol{u}^B$$

it must follow that $\boldsymbol{R}_A^B \left(\boldsymbol{R}_A^B\right)^\top = \boldsymbol{I}$ and therefore $\left(\boldsymbol{R}_A^B\right)^\top = \left(\boldsymbol{R}_A^B\right)^{-1}$ implying that the rows are mutually orthogonal with the same applying to the columns. Generally for coordinate transformations $\boldsymbol{I}$ represents the identity transform and $\boldsymbol{R}_A^B = \boldsymbol{I} \implies \mathcal{F}^A = \mathcal{F}^B$. Using some basic linear algebra we see that

$$\det(\boldsymbol{I}) = \det\left(\boldsymbol{R}_A^B \left(\boldsymbol{R}_A^B\right)^{-1}\right) = \det\left(\boldsymbol{R}_A^B\right) \det\left(\left(\boldsymbol{R}_A^B\right)^\top\right) = \det\left(\boldsymbol{R}_A^B\right)^2 = 1$$

which implies that $\det\left(\boldsymbol{R}_A^B\right) = \pm 1$. A coordinate transformation corresponds to the case $\det\left(\boldsymbol{R}_A^B\right) = 1$ which can easily be seen from using the definition of the inverse

$$\boldsymbol{u}^A = \left(\boldsymbol{R}_A^B\right)^\top \boldsymbol{u}^B = \frac{1}{\det\left(\boldsymbol{R}_A^B\right)} \mathrm{Adj}\left(\boldsymbol{R}_A^B\right) \boldsymbol{u}^B = \frac{1}{\det\left(\boldsymbol{R}_A^B\right)} \boldsymbol{u}^A$$

which implies that $\det\left(\boldsymbol{R}_B^A\right) = 1$, the fact that $\mathrm{Adj}\left(\boldsymbol{R}_A^B\right) \boldsymbol{u}^B = \boldsymbol{u}^A$ can easily be seen by deriving the cofactors and using that $\boldsymbol{i} = \boldsymbol{j} \times \boldsymbol{k}$, $\boldsymbol{j} = \boldsymbol{k} \times \boldsymbol{i}$, $\boldsymbol{k} = \boldsymbol{i} \times \boldsymbol{j}$.

2.4 Attitude representations

The attitude of a rigid-body relative to some reference frame $\mathcal{F}^A$ can be represented as a coordinate transformation between a reference frame fixed to the rigid-body $\mathcal{F}^B$ and $\mathcal{F}^A$. In Satellite applications the reference frame fixed to the satellite usually called the body frame denoted $\mathcal{F}^b$, and the other reference frame is either an inertial reference frame like the ECI frame, or some non-inertial frame like the orbit frame or ECEF frame. In other words when working with satellites it is important to be comfortable with using several reference frames both inertial and non-inertial. The are several ways to represent a coordinate transformation but in this course we focus on the most common ones

[vi] Assuming that $\boldsymbol{I} = I_x\boldsymbol{i} + I_y\boldsymbol{j} + I_z\boldsymbol{k}$, $\boldsymbol{J} = J_x\boldsymbol{i} + J_y\boldsymbol{j} + J_z\boldsymbol{k}$, $\boldsymbol{K} = K_x\boldsymbol{i} + K_y\boldsymbol{j} + K_z\boldsymbol{k}$.

[vii] Yes, I'm being relaxed with the notation since $\boldsymbol{I}$, $\boldsymbol{J}$ and $\boldsymbol{K}$ are expressed in $\mathcal{F}^A$ and previously it was $\boldsymbol{i}$, $\boldsymbol{j}$ and $\boldsymbol{k}$ that was expressed in $\mathcal{F}^B$. However, the math works out since coordinate transformations preserve direction and magnitude of vectors and therefore the scalar/dot product between them, $\boldsymbol{u}^A \bullet \boldsymbol{v}^A = \boldsymbol{u}^B \bullet \boldsymbol{v}^B$.

- Euler angles
- Direction Cosine Matrices
- Axis-Angle
- Quaternions

These are just different ways to paramaterize a coordinate transform which from the previous section was a 3 by 3 matrix $\boldsymbol{R}$ with the two properties

$$\begin{aligned} &1.\ \boldsymbol{R}^{-1} = \boldsymbol{R}^T \\ &2.\ \det(\boldsymbol{R}) = 1 \end{aligned}$$

The set of all these matrices is called the Special Orthogonal Group of dimension 3

$$SO(3) := \left\{ \boldsymbol{R} \in \mathbb{R}^{3\times 3}; \boldsymbol{R}^{-1} = \boldsymbol{R}^\top; \det \boldsymbol{R} = 1 \right\}.$$

The attitude kinematics are essentially how these representations relate to the angular velocity of the rigid-body. Since the body frame is fixed to the satellite this is equivalent to the angular velocity of $\mathcal{F}^b$ relative to for instance $\mathcal{F}^i$. We will denote the angular velocity by $\boldsymbol{\omega}_{ib}^b = \begin{bmatrix} p & q & r \end{bmatrix}^\top$.

2.5 Euler Angles

Euler angles parameterizes the rotation matrix using three angles α, β and γ where each angle is used to construct a intermediary rotation matrix

$$\begin{aligned} \boldsymbol{R}_1(\vartheta_1) &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\vartheta_1 & -\sin\vartheta_1 \\ 0 & \sin\vartheta_1 & \cos\vartheta_1 \end{bmatrix} \\ \boldsymbol{R}_2(\vartheta_2) &= \begin{bmatrix} \cos\vartheta_2 & 0 & \sin\vartheta_2 \\ 0 & 1 & 0 \\ -\sin\vartheta_2 & 0 & \cos\vartheta_2 \end{bmatrix} \\ \boldsymbol{R}_3(\vartheta_3) &= \begin{bmatrix} \cos\vartheta_3 & -\sin\vartheta_3 & 0 \\ \sin\vartheta_3 & \cos\vartheta_3 & 0 \\ 0 & 0 & 1 \end{bmatrix} \end{aligned}$$

and then by multiplying these matrices together in a specific order we get our final rotation matrix. The angles together with the order of multiplication defines an Euler sequence.

Example 1: Roll ϕ, pitch θ, yaw ψ sequence from body frame to ECI frame

$$\boldsymbol{R}_b^i = \boldsymbol{R}_3(\psi)\boldsymbol{R}_2(\theta)\boldsymbol{R}_1(\phi)$$

Example 2: Classical Euler sequence from perifocal frame to ECI frame

$$\boldsymbol{R}_p^i = \boldsymbol{R}_3(\Omega)\boldsymbol{R}_2(i)\boldsymbol{R}_3(\omega)$$

The form of the attitude kinematics will depend on the Euler sequence used, but typically the roll, pitch, yaw sequence is used to represent attitude of rigid-bodies. Therefore the attitude kinematics for this sequence is

$$\begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} = \begin{bmatrix} 1 & \sin\phi\tan\theta & \cos\phi\tan\theta \\ 0 & \cos\phi & -\sin\phi \\ 0 & \frac{\sin\phi}{\cos\theta} & \frac{\cos\phi}{\cos\theta} \end{bmatrix} \begin{bmatrix} p \\ q \\ r \end{bmatrix} = \boldsymbol{W}\boldsymbol{\omega}_{ib}^{b}$$

2.6 Direction Cosine Matrices

Instead of using three angles to parameterize the rotation matrix we can use nine direction cosines, which are the projections of $\boldsymbol{i}$, $\boldsymbol{j}$ and $\boldsymbol{k}$ expressed in $\mathcal{F}^i$ onto $\boldsymbol{I}$, $\boldsymbol{J}$ and $\boldsymbol{K}$

$$\boldsymbol{R}_b^i = \begin{bmatrix} c\theta_{Ii} & c\theta_{Ij} & c\theta_{Ik} \\ c\theta_{Ji} & c\theta_{Jj} & c\theta_{Jk} \\ c\theta_{Ki} & c\theta_{Kj} & c\theta_{Kk} \end{bmatrix}$$

The attitude kinematics when using DCMs is given by

$$\dot{\boldsymbol{R}}_b^i = S\left(\boldsymbol{\omega}_{ib}^i\right)\boldsymbol{R}_b^i = \boldsymbol{R}_b^i S\left(\boldsymbol{\omega}_{ib}^b\right)$$

where

$$S\left(\boldsymbol{\omega}_{ib}^i\right) = \begin{bmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{bmatrix}, \qquad S\left(\boldsymbol{\omega}_{ib}^b\right) = \begin{bmatrix} 0 & -r & q \\ r & 0 & -p \\ -q & p & 0 \end{bmatrix}$$

In the attitude kinematics $\dot{\boldsymbol{R}}_b^i$ is not a rotation matrix, for instance when $\boldsymbol{\omega}_{ib}^b = \boldsymbol{0}$ so is $\dot{\boldsymbol{R}}_b^i$.

2.7 Axis-Angle

Euler's rotation theorem states that between two orientations there always exists an axis such that a rotation about that axis with an angle θ will align the orientations. This axis called the axis the Euler axis. The Axis-Angle representation is just the direction of the Euler axis represented as a vector $\boldsymbol{u}$ multiplied by the rotation angle θ

$$\boldsymbol{\theta} = \theta \cdot \boldsymbol{u}$$

Usually this representation is not used by itself, but forms a convenient way to construct quaternions. Another way to express the Euler's theorem is that for any rotation matrix $\boldsymbol{R} \in SO(3)$ there exist a unique unit vector $\boldsymbol{u}$ such that

$$\boldsymbol{R}\boldsymbol{u} = \boldsymbol{u}$$

in other words the coordinates of $\boldsymbol{u}$ is invariant under coordinate transformation $\boldsymbol{R}$. We see that from linear algebra this $\boldsymbol{u}$ is an eigenvector of $\boldsymbol{R}$ with eigenvalue 1 and therefore given a rotation matrix we can solve the eigenvector problem to

find the Euler axis and from there find the axis-angle representation of $\boldsymbol{R}$. If we are given two vectors $\boldsymbol{a}$, $\boldsymbol{b}$ the axis-angle that can rotate $\boldsymbol{a}$ to $\boldsymbol{b}$ can be found as

$$\theta = \arccos\left(\frac{\boldsymbol{a} \bullet \boldsymbol{b}}{\|\boldsymbol{a}\| \cdot \|\boldsymbol{b}\|}\right)$$
$$\boldsymbol{u} = \frac{\boldsymbol{a} \times \boldsymbol{b}}{\|\boldsymbol{a} \times \boldsymbol{b}\|}$$

However, two vectors can only specify two of the three coordinates needed to fully specifiy a coordinate transformation. Therefore if you have two reference frames A and B with coordinate transformation $\boldsymbol{R}_B^A$ and vectors $\boldsymbol{a}^B$ and $\boldsymbol{a}^A$ then

$$\left(\boldsymbol{I} - \boldsymbol{R}_B^A\right)\boldsymbol{u} = \boldsymbol{0} \rightarrow \theta\boldsymbol{u} \neq \arccos\left(\frac{\boldsymbol{a}^B \bullet \boldsymbol{a}^A}{\|\boldsymbol{a}^B\| \cdot \|\boldsymbol{a}^A\|}\right) \cdot \frac{\boldsymbol{a}^B \times \boldsymbol{a}^A}{\|\boldsymbol{a}^B \times \boldsymbol{a}^A\|}$$

In other words the axis-angle constructed from $\boldsymbol{a}^A$ and $\boldsymbol{a}^B$ will transform the coordinates $\boldsymbol{a}^B$ to $\boldsymbol{a}^A$ but given another vector $\boldsymbol{b}^B$ then using this constructed axis-angle to rotate $\boldsymbol{b}^B$ generally won't give $\boldsymbol{b}^A$. It is possible to construct another axis-angle using $\boldsymbol{b}^B$ and $\boldsymbol{b}^A$ such that the composition of these two axis-angles actually do give the full coordinate transformation. This idea of constructing a coordinate transformation from two sets of vectors $\boldsymbol{a}^B, \boldsymbol{b}^B$ and $\boldsymbol{a}^A, \boldsymbol{b}^A$ is the basis of deterministic attitude determination methods, which we will come back to.

2.8 Quaternions

A quaternion can be written as a 4D vector

$$\boldsymbol{q} = \begin{bmatrix} q_0 \\ \boldsymbol{q}_v \end{bmatrix} \in \mathbb{R}^4$$

where q_0 is usually called the real part and $\boldsymbol{q}_v$ the vector part. The conjugate of a quaternion is defined as

$$\boldsymbol{q}^* = \begin{bmatrix} q_0 \\ -\boldsymbol{q}_v \end{bmatrix}$$

while the norm is

$$\|\boldsymbol{q}\| = \sqrt{q_0^2 + \boldsymbol{q}_v^\top \boldsymbol{q}_v}$$

Together these two operations define the inverse of a quaternion

$$\boldsymbol{q}^{-1} = \frac{1}{\|\boldsymbol{q}\|^2}\boldsymbol{q}^*$$

We can define addition/subtraction and multiplication between two quaternions $\boldsymbol{q}, \boldsymbol{r} \in \mathbb{R}^4$ as

$$\boldsymbol{q} \pm \boldsymbol{r} = \begin{bmatrix} q_0 \pm r_0 \\ \boldsymbol{q}_v \pm \boldsymbol{r}_v \end{bmatrix}, \qquad \boldsymbol{q} \otimes \boldsymbol{r} = \begin{bmatrix} q_0 r_0 - \boldsymbol{q}_v^\top \boldsymbol{r}_v \\ q_0 \boldsymbol{r}_v + r_0 \boldsymbol{r}_v + \boldsymbol{q}_v \times \boldsymbol{r}_v \end{bmatrix}$$

A Unit Quaternion is just a quaternion with unit length, just like we would define a unit vector as a vector with unit length. In other words

$$\boldsymbol{q} \in \mathbb{R}^4 \text{ with } \|\boldsymbol{q}\| = 1$$

and an immediate consequence of this is that the inverse of a unit quaternion is equal to its conjugate $\boldsymbol{q}^{-1} = \boldsymbol{q}^*$. Unit quaternions can be used as coordinate transformations by multiplying them with vector quaternions. A vector quaternion is just a quaternion with zero real part. Let's say we have vector $\boldsymbol{p}^b \in \mathbb{R}^3$, it can be written as a vector quaternion

$$\boldsymbol{p}^b \mapsto \begin{bmatrix} 0 \\ \boldsymbol{p}^b \end{bmatrix}$$

We can then multiply it with a unit quaternion $\boldsymbol{q}_{ib}$ to perform a coordinate transformation

$$\boldsymbol{p}^i = \boldsymbol{q}_{ib} \otimes \boldsymbol{p}^b \otimes \boldsymbol{q}_{ib}^*$$

where the resulting $\boldsymbol{p}^i$ is also a vector quaternion and can therefore be easily converted to a vector[(viii)]

$$\begin{bmatrix} 0 \\ \boldsymbol{p}^i \end{bmatrix} \mapsto \boldsymbol{p}^i$$

The attitude kinematics with quaternions is

$$\dot{\boldsymbol{q}}_{ib} = \frac{1}{2}\boldsymbol{\omega}_{ib}^i \otimes \boldsymbol{q}_{ib} = \frac{1}{2}\boldsymbol{q}_{ib} \otimes \boldsymbol{\omega}_{ib}^b$$

where $\dot{\boldsymbol{q}}_{ib}$ is a quaternion, i.e. it is in general not unit length.

[(viii)] In implementations this conversion between vectors and vector quaternions is usually handled in the software so that we don't have to think about it.

2.9 Conversions between attitude representations

Some conversions between different SO(3) representations are given below

2.9.1 Axis-angle to unit quaternions

Given $\theta \boldsymbol{u}$, $\|\boldsymbol{u}\| = 1$

$$\begin{bmatrix} q_0 \\ \boldsymbol{q}_v \end{bmatrix} = \begin{bmatrix} \cos\frac{\theta}{2} \\ \boldsymbol{u}\sin\frac{\theta}{2} \end{bmatrix}, \qquad \|\boldsymbol{q}\| = 1$$

2.9.2 Axis-angle to DCM

Given $\theta \boldsymbol{u}$, $\|\boldsymbol{u}\| = 1$

$$\boldsymbol{R} = \boldsymbol{I} + \sin\theta S(\boldsymbol{u}) + (1 - \cos\theta) S(\boldsymbol{u})^2$$

2.9.3 Unit quaternion to DCM

Given $\boldsymbol{q} = \begin{bmatrix} q_0 & q_1 & q_2 & q_3 \end{bmatrix}^\top$

$$\boldsymbol{R}(\boldsymbol{q}) = \begin{bmatrix} q_0^2 + q_1^2 - q_2^2 - q_3^2 & 2(q_1q_2 + q_0q_3) & 2(q_1q_3 - q_0q_2) \\ 2(q_1q_2 - q_0q_3) & q_0^2 - q_1^2 + q_2^2 - q_3^2 & 2(q_2q_3 + q_0q_1) \\ 2(q_1q_3 + q_0q_2) & 2(q_2q_3 - q_0q_1) & q_0^2 - q_1^2 - q_2^2 + q_3^2 \end{bmatrix}$$

2.9.4 DCM to unit quaternion

Given $\boldsymbol{R} \in SO(3)$ with elements r_{ij}, there exist several algorithms to compute a unit quaternion. We present three well known algorithms in Appendix A.

2.9.5 Roll, pitch, yaw to DCM

Given roll, pitch, yaw (ϕ, θ, ψ) angles

$$\begin{aligned} \boldsymbol{R}_b^i &= \boldsymbol{R}_3(\psi)\boldsymbol{R}_2(\theta)\boldsymbol{R}_1(\phi) \\ &= \begin{bmatrix} c\theta c\psi & s\phi s\theta c\psi - c\phi s\psi & c\phi s\theta c\psi + s\phi s\psi \\ c\theta s\psi & s\phi s\theta s\psi + c\phi c\psi & c\phi s\theta s\psi - s\phi c\psi \\ -s\theta & s\phi c\theta & c\phi c\theta \end{bmatrix} \end{aligned}$$

2.9.6 DCM to roll, pitch, yaw

Given $\boldsymbol{R} \in SO(3)$ with elements r_{ij}, we can extract any Euler sequence from it. To get the roll, pitch, yaw sequence we do

$$\begin{aligned} \phi &= \arctan 2\left(r_{23}, r_{33}\right) \\ \theta &= \arcsin\left(-r_{13}\right) \\ \psi &= \arctan 2\left(r_{12}, r_{11}\right) \end{aligned}$$

2.9.7 Roll, pitch, yaw to unit quaternion

Given roll, pitch, yaw (ϕ, θ, ψ) angles

$$\boldsymbol{q}_{ib} = \begin{bmatrix} \cos\frac{\psi}{2} \\ 0 \\ 0 \\ \sin\frac{\psi}{2} \end{bmatrix} \otimes \begin{bmatrix} \cos\frac{\theta}{2} \\ 0 \\ \sin\frac{\theta}{2} \\ 0 \end{bmatrix} \otimes \begin{bmatrix} \cos\frac{\phi}{2} \\ \sin\frac{\phi}{2} \\ 0 \\ 0 \end{bmatrix}$$

2.9.8 Unit quaternion to roll, pitch, yaw

Given $\boldsymbol{q} = \begin{bmatrix} q_0 & q_1 & q_2 & q_3 \end{bmatrix}^\top$

$$\begin{aligned} \phi &= \arctan 2\left(2\left(q_0 q_1 + q_2 q_3\right), q_0^2 + q_3^2 - q_1^2 - q_2^2\right) \\ \theta &= \arcsin\left(2\left(q_0 q_2 - q_1 q_3\right)\right) \\ \psi &= \arctan 2\left(2\left(q_0 q_3 + q_1 q_2\right), q_0^2 + q_1^2 - q_2^2 - q_3^2\right) \end{aligned}$$

2.9.9 Composition of coordinate transforms

Coordinate transformations $SO(3)$ is a subset of $\mathbb{R}^{3\times 3}$ and form a group under multiplication[ix] with identity element $\boldsymbol{I} \in SO(3)$. Therefore for $\boldsymbol{A}, \boldsymbol{B} \in SO(3)$, $\boldsymbol{AB} \in SO(3)$, $\boldsymbol{AI} = \boldsymbol{A} \in SO(3)$. The multiplicative inverse of a coordinate transformation $\boldsymbol{A}$ is given by its transpose $\boldsymbol{A}\boldsymbol{A}^\top = \boldsymbol{A}^\top\boldsymbol{A} = \boldsymbol{I}$. $SO(3)$ is not closed under addition such that $\boldsymbol{A} + \boldsymbol{B} \notin SO(3)$ and scalar multiplication $a \in \mathbb{R}$, $a\boldsymbol{A} \notin SO(3)$. Therefore we can only multiply coordinate transformations together to get a new coordinate transformation, this forms the basis for combining them (composition).

If we have three reference frames $\mathcal{F}^A$, $\mathcal{F}^B$ and $\mathcal{F}^C$ with coordinate transformations $\boldsymbol{R}_A^B$ and $\boldsymbol{R}_B^C$ then the coordinate transformation from $\mathcal{F}^A$ to $\mathcal{F}^C$ can be found through multiplying the transforms

$$\boldsymbol{R}_A^C = \boldsymbol{R}_{\color{red}B}^{\color{blue}C} \cdot \boldsymbol{R}_{\color{red}A}^{\color{blue}B}$$

For the result to make sense we have to adhere to some composition rules for coordinate transformations. We highlight these rules with the color coding where frames highlighted in red need to be the same for both coordinate transforms. Blue denotes don't care so they can be any reference frame. Remember that in our notation $\boldsymbol{R}_{\text{from}}^{\text{to}}$, such that the above example can be read as **'From A to B then from B to C'** which through composition is equivalent to **'From A to C'**. We know that $\boldsymbol{R}_C^B = \left(\boldsymbol{R}_B^C\right)^\top$ such that the above can also be written as

$$\boldsymbol{R}_A^C = \left(\boldsymbol{R}_{\color{blue}C}^{\color{red}B}\right)^\top \cdot \boldsymbol{R}_{\color{red}A}^{\color{blue}B}$$

if we are given $\boldsymbol{R}_C^B$ instead of $\boldsymbol{R}_B^C$.

With unit quaternions we have the exact same rules. Unit quaternions is a subset of quaternions and form a group under quaternion multiplication $\otimes$ with identity element $\boldsymbol{q}_I = \begin{bmatrix} 1 & 0 & 0 & 0 \end{bmatrix}^\top$. Unit quaternions is also not closed under addition and scalar multiplication. Just like multiplication is used to

[ix] In notation we use either $\cdot$ or nothing to denote a multiplication, $\boldsymbol{A} \cdot \boldsymbol{B} = \boldsymbol{AB}$

combine rotation matrices, quaternion multiplication is used to combine unit quaternions with the same rules just with quaternion notation where we use $\boldsymbol{q}_{\text{tofrom}}$. Therefore

$$\boldsymbol{q}_{CA} = \boldsymbol{q}_{CB} \otimes \boldsymbol{q}_{BA} = \boldsymbol{q}^*_{BC} \otimes \boldsymbol{q}_{BA}$$

with $\boldsymbol{q}_{BC} = \boldsymbol{q}^*_{CB}$. This notation is very convenient since it tells us how to properly combine and apply the to transform vectors

$$\boldsymbol{u}^C = \boldsymbol{q}_{CB} \otimes \boldsymbol{u}^B \otimes \boldsymbol{q}^*_{CB} = \boldsymbol{q}_{CB} \otimes \boldsymbol{u}^B \otimes \boldsymbol{q}_{BC}$$

so the inner frame have to agree with the frame of the vector and the resulting vector is given in the outer frame.

We also want to be able to combine angular velocities which is done with addition/subtraction. Because angular velocities are just elements of $\mathbb{R}^3$ which form a vector space with addition and scalar multiplication. The inverse of an angular velocity is then just its negation for instance $\boldsymbol{\omega}^B_{CA} = -\boldsymbol{\omega}^B_{AC}$, such that $\boldsymbol{\omega}^B_{CA} + \boldsymbol{\omega}^B_{AC} = \mathbf{0}$. The rules we need to follow are similar to the coordinate transformations

$$\begin{aligned}\boldsymbol{\omega}^B_{CA} &= \boldsymbol{\omega}^B_{CB} + \boldsymbol{\omega}^B_{BA} \\ \boldsymbol{\omega}^C_{BC} &= \boldsymbol{\omega}^C_{BA} - \boldsymbol{\omega}^C_{CA}\end{aligned}$$

Essentially they need to be expressed in the same frame and the red subscripts have to be the same.

2.10 Attitude Dynamics

The attitude dynamics is captured by the angular acceleration of the rigid-body and be expressed through Euler's moment equation

$$\dot{\boldsymbol{h}}^b = \boldsymbol{\tau}^b - \boldsymbol{\omega}^b_{ib} \times \boldsymbol{h}^b$$

since $\boldsymbol{h}^b = \boldsymbol{J}^b \boldsymbol{\omega}^b_{ib}$ and $\dot{\boldsymbol{J}}^b = \mathbf{0}$ we get

$$\boldsymbol{J}^b \dot{\boldsymbol{\omega}}^b_{ib} = \boldsymbol{\tau}^b - \boldsymbol{\omega}^b_{ib} \times \boldsymbol{J}^b \boldsymbol{\omega}^b_{ib}$$

where $\boldsymbol{J}^b$ is the inertia matrix expressed in the body frame and $\boldsymbol{\tau}^b$ are external torques acting on the rigid-body.

3 Simulating Satellite Attitude Kinematics and Dynamics

3.1 Numeric integration of coordinate transformations

Depending on the attitude representation we can have three different sets of differential equations, although the dynamics is the same in all three cases

Attitude representation	Kinematics	Dynamics
Euler Angles	$\begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} = \boldsymbol{W}\boldsymbol{\omega}_{ib}^b$	$\boldsymbol{J}^b\dot{\boldsymbol{\omega}}_{ib}^b = \boldsymbol{\tau}^b - \boldsymbol{\omega}_{ib}^b \times \boldsymbol{J}^b\boldsymbol{\omega}_{ib}^b$
Direction Cosine Matrices	$\dot{\boldsymbol{R}}_b^i = \boldsymbol{R}_b^i S\left(\boldsymbol{\omega}_{ib}^b\right)$	$\boldsymbol{J}^b\dot{\boldsymbol{\omega}}_{ib}^b = \boldsymbol{\tau}^b - \boldsymbol{\omega}_{ib}^b \times \boldsymbol{J}^b\boldsymbol{\omega}_{ib}^b$
Unit Quaternions	$\dot{\boldsymbol{q}}_{ib} = \frac{1}{2}\boldsymbol{q}_{ib} \otimes \boldsymbol{\omega}_{ib}^i$	$\boldsymbol{J}^b\dot{\boldsymbol{\omega}}_{ib}^b = \boldsymbol{\tau}^b - \boldsymbol{\omega}_{ib}^b \times \boldsymbol{J}^b\boldsymbol{\omega}_{ib}^b$

When working with Euler angles we can simply treat them like we would any other differential equation. However, because of the singularities in $\boldsymbol{W}$ when $\theta = \pm\frac{\pi}{2}$ we will have to explicitly take care of these cases. Since satellites might orient themselves arbitrarily, it is not common to use Euler angles for attitude simulation.

When working with attitude kinematics using DCMs or unit quaterions care must be take when updating them. As an example updating a DCM/unit quaternion with forward Euler would be

$$\boldsymbol{R}_{k+1} = \boldsymbol{R}_k + h \cdot \dot{\boldsymbol{R}}$$
$$\boldsymbol{q}_{k+1} = \boldsymbol{q}_k + h \cdot \dot{\boldsymbol{q}}$$

but we know that addition with DCMs/unit quaternions does not result in DCMs/unit quaternions. Furthermore, we have that in general $h \cdot \dot{\boldsymbol{R}} \notin SO(3)$. Therefore the result of the update $\boldsymbol{R}_{k+1} \notin SO(3)$ and $\|\boldsymbol{q}_{k+1}\| \neq 1$ and it might seem like we can't use regular numeric solvers for attitude kinematics. Luckily the fix is fairly straightforward and we don't have to modify our numeric solvers. The fix is to take $\boldsymbol{R}_{k+1}$ or $\boldsymbol{q}_{k+1}$ and essentially reproject them back into $SO(3)$ or the set of unit quaternions.

With unit quaternions this process is very simple, after the update we just normalize $\boldsymbol{q}_{k+1}$ making it a unit quaternion. With DCMs we need to reorthogonalize the matrix. There are a few possible ways to do this, one approach is to first compute the SVD of $\boldsymbol{R}_{k+1} = \boldsymbol{U}\boldsymbol{S}\boldsymbol{V}^\top$ then compute the orthogonalized $\boldsymbol{R}_{k+1}$ as $\boldsymbol{U}\boldsymbol{V}^\top$ this reorthogonalized $\boldsymbol{R}_{k+1}$ then has the properties $\det\left(\boldsymbol{R}_{k+1}\right) = 1$ and $\boldsymbol{R}_{k+1}\boldsymbol{R}_{k+1}^\top = \boldsymbol{I}$. An alternative is to use Bar's algorithm in Appendix A to construct a unit quaternion from $\boldsymbol{R}_{k+1}$ and then convert it back into a DCM to give the reorthogonalized $\boldsymbol{R}_{k+1}$.

Since quaternions have no kinematic singularities, fewer coordinates to update relative to DCMs and is generally easier to numerically integrate and reproject, it is commonly used for the attitude representation in satellite attitude simulators. This is why we mainly focus on them in this course for attitude simulation purposes. However, when working with methods for Attitude Determination and Control we will see that some of them use might Euler angles or

DCMs and therefore we still need to support conversion between the different attitude representations.

Also note the when using RK4 a total of 5 reprojections is needed, one for each of the four intermediary points and again after the final update step. This can still be done without modifying the RK4 implementation. We just do the reprojections for the intermediary points inside the provided 'f' function and the reprojection for the final update step is just done on the returned state from 'step_RK4'. This idea is shown in Listing 1 in Appendix B.

As a small sidenote we've seen that for simulating attitude kinematics we can use regular numeric integration, but we might wonder if there are other ways to this without needing the reprojection step. In fact there are and they are called geometric integrators[(x)]. These methods are usually based on

$$\boldsymbol{R}_{k+1} = \boldsymbol{R}_k \cdot e^{hS(\boldsymbol{\omega})}$$
$$\boldsymbol{q}_{k+1} = \boldsymbol{q}_k \otimes e^{\frac{1}{2}h\boldsymbol{\omega}}$$

and we will briefly revisit them later in the course.

3.2 Simulating attitude kinematics and dynamics with inputs

Depending on the attitude representation we can have three different sets of equations, although the dynamics is the same in all three cases

Attitude representation	Kinematics	Dynamics
Euler Angles	$\begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} = \boldsymbol{W}\boldsymbol{\omega}_{ib}^b$	$\boldsymbol{J}^b\dot{\boldsymbol{\omega}}_{ib}^b = \boldsymbol{\tau}^b - \boldsymbol{\omega}_{ib}^b \times \boldsymbol{J}^b\boldsymbol{\omega}_{ib}^b$
Direction Cosine Matrices	$\dot{\boldsymbol{R}}_b^i = \boldsymbol{R}_b^i S\left(\boldsymbol{\omega}_{ib}^b\right)$	$\boldsymbol{J}^b\dot{\boldsymbol{\omega}}_{ib}^b = \boldsymbol{\tau}^b - \boldsymbol{\omega}_{ib}^b \times \boldsymbol{J}^b\boldsymbol{\omega}_{ib}^b$
Unit Quaternions	$\dot{\boldsymbol{q}}_{ib} = \frac{1}{2}\boldsymbol{q}_{ib} \otimes \boldsymbol{\omega}_{ib}^i$	$\boldsymbol{J}^b\dot{\boldsymbol{\omega}}_{ib}^b = \boldsymbol{\tau}^b - \boldsymbol{\omega}_{ib}^b \times \boldsymbol{J}^b\boldsymbol{\omega}_{ib}^b$

With unit quaternnios the system can then be written in the standard first order ordinary differential equation form with inputs $f(t, \boldsymbol{x}, \boldsymbol{u})$

$$\boldsymbol{x} = \begin{bmatrix} \boldsymbol{q}_{ib} \\ \boldsymbol{\omega}_{ib}^b \end{bmatrix}, \qquad \dot{\boldsymbol{x}} = \begin{bmatrix} \frac{1}{2}\boldsymbol{q}_{ib} \otimes \boldsymbol{\omega}_{ib}^i \\ \left(\boldsymbol{J}^b\right)^{-1}\left(\boldsymbol{\tau}^b - \boldsymbol{\omega}_{ib}^b \times \boldsymbol{J}^b\boldsymbol{\omega}_{ib}^b\right) \end{bmatrix}$$

Although the equations have the form $f(t, \boldsymbol{x}, \boldsymbol{u})$ with $\boldsymbol{\tau}^b$ as the input we can still use our solvers that assume $f(t, \boldsymbol{x})$. The trick is that since we provide our own f function, we can make it so that it has access to $\boldsymbol{u}$ so that it doesn't need to be passed as an argument. This idea is shown in Listing 1 in Appendix B.

[(x)] We should note that they don't require reprojection in theory, but in practice it still needs to be done just less often, which in some applications can be a performance gain.

3.3 Assignment 4.1: Torque free motion of a Rigid-Body

Consider a rigid-body with no external torques acting on it. Then the attitude dynamics reduces to

$$\dot{\boldsymbol{\omega}}_{ib}^{b} = \left(\boldsymbol{J}^{b}\right)^{-1}\left(\boldsymbol{J}^{b}\boldsymbol{\omega}_{ib}^{b} \times \boldsymbol{\omega}_{ib}^{b}\right)$$

if we assume that $\boldsymbol{J}^b$ is a diagonal matrix with A, B and C along the diagonal then expanding the equation into component form gives

$$\begin{aligned} A\dot{p} - (B - C)qr &= 0 \\ B\dot{q} - (C - A)pr &= 0 \\ C\dot{r} - (A - B)pq &= 0 \end{aligned}$$

Consider the case with initial values $p = 0$, $q = q_0$, $r = r_0$, $r_0 > q_0$ and parameters $A = 1$, $B = 2$ and $C = 1$, then at the initial time we have

$$\begin{aligned} \dot{p} - q_0 r_0 &= 0 \\ \dot{q} &= 0 \\ \dot{r} &= 0 \end{aligned}$$

which will cause p to increase. But as p increases the equations take the form

$$\begin{aligned} \dot{p} - q_0 r &= 0 \\ \dot{q} &= 0 \\ \dot{r} + p q_0 &= 0 \end{aligned}$$

such that any increase in p will cause a decrease in r. Eventually r will become negative such that p will start to decrease while r increases. The total angular momentum is preserved and the system has a periodic motion, where it first spins counter-clockwise around the body z-axis, then it will switch direction while spinning counter-clockwise around the body x-axis, then it will switch the spinning direction about the x-axis as the motion repeats. This behaviour of trading angular momentum between the roll, pitch and yaw axes can seen create some interesting motions.

Assignment 4.1

1. Extend simutils.py with functions:

   quaternion_to_dcm

   axis_angle_to_dcm

   dcm_to_quaternion (use Shepperd's algorithm)

   euler_to_quaternion

   quaternion_to_euler

   dcm_to_euler

   euler_to_dcm

2. Extend the sat_lib.py with a RigidBody class, that has class members orientation, angular velocity, inertia matrix and torque. Let the internal attitude representation be quaternions. Let the class have the following methods

__init__: constructs a rigid-body with a given orientation, angular velocity and inertia matrix.

update: Takes current time, time step and external torques as input and computes the updated state.

f: Implements $f(t, \boldsymbol{x})$, used by the numerical solvers.

See Listing 1 in Appendix B for an example implementation of a dynamic system that includes state reprojection and inputs.

3. Simulate a rigid-body using RK4 with the parameters:
   t_0= 0, t_e= 100, t_step= 0.01, speed_factor = 1, anim_dt = 1/25, scale_factor= 1.

$$
\begin{aligned}
\boldsymbol{q}_{ib}(0) &= \begin{bmatrix} 1 & 0 & 0 & 0 \end{bmatrix}^\top \\
\boldsymbol{\omega}_{ib}^b(0) &= \begin{bmatrix} 0 & 0 & 5 \end{bmatrix}^\top \\
\boldsymbol{\tau}^b &= \begin{bmatrix} 0 & 0 & 0 \end{bmatrix}^\top \\
\boldsymbol{J}^b &= \begin{bmatrix} 2 & 1 & 0 \\ 1 & 10 & 0.1 \\ 0 & 0.1 & 2.5 \end{bmatrix}
\end{aligned}
$$

3.4 Assignment 4.2: Satellite Attitude Control

Consider the satellite

$$\dot{\boldsymbol{q}}_{ib} = \frac{1}{2}\boldsymbol{q}_{ib} \otimes \boldsymbol{\omega}_{ib}^{i}$$
$$\boldsymbol{J}^b \dot{\boldsymbol{\omega}}_{ib}^{b} = \boldsymbol{\tau}^b - \boldsymbol{\omega}_{ib}^{b} \times \boldsymbol{J}^b \boldsymbol{\omega}_{ib}^{b}$$

which describe the attitude kinematics and dynamics of a satellite with respect to the ECI frame. Usually we want control the satellite attitude relative to the orbit frame $\mathcal{F}^o$, which was defined as

$$\boldsymbol{R}_o^i = \begin{bmatrix} \frac{\boldsymbol{h}^i \times \boldsymbol{r}^i}{\|\boldsymbol{h}^i\|\|\boldsymbol{r}^i\|} & -\frac{\boldsymbol{h}^i}{\|\boldsymbol{h}^i\|} & -\frac{\boldsymbol{r}^i}{\|\boldsymbol{r}^i\|} \end{bmatrix}$$

with $\boldsymbol{h}^i = \boldsymbol{r}^i \times \dot{\boldsymbol{r}}^i$. It can be converted to $\boldsymbol{q}_{ob}$ using for instance Reynold's algorithm[(xi)]. The kinematics can be found by evaluating the derivative of $\boldsymbol{R}_o^i$ which leads to

$$\boldsymbol{\omega}_{io}^{i} = \frac{\dot{\boldsymbol{r}}^i \times \boldsymbol{r}^i}{r^2} = -\frac{\boldsymbol{h}^i}{r^2}, \qquad \dot{\boldsymbol{\omega}}_{io}^{i} = -\frac{2\boldsymbol{r}^\top \dot{\boldsymbol{r}}}{r^2}\boldsymbol{\omega}_{io}^{i} = \frac{2\boldsymbol{r}^{i\top}\dot{\boldsymbol{r}}}{r^4}\boldsymbol{h}^i$$

The attitude with respect to the orbit frame and the corresponding attitude kinematics is then

$$\begin{aligned}
\boldsymbol{q}_{ob} &= \boldsymbol{q}_{io}^* \otimes \boldsymbol{q}_{ib} \\
\dot{\boldsymbol{q}}_{ob} &= \dot{\boldsymbol{q}}_{io}^* \otimes \boldsymbol{q}_{ib} + \boldsymbol{q}_{io}^* \otimes \dot{\boldsymbol{q}}_{ib} \\
&= \frac{1}{2}\boldsymbol{q}_{ob} \otimes \left(\boldsymbol{\omega}_{ib}^{b} - \boldsymbol{q}_{ob}^* \otimes \boldsymbol{\omega}_{io}^{o} \otimes \boldsymbol{q}_{ob}\right) = \frac{1}{2}\boldsymbol{q}_{ob} \otimes \boldsymbol{\omega}_{ob}^{b}
\end{aligned}$$

which result in the dynamics

$$\begin{aligned}
\boldsymbol{J}^b \dot{\boldsymbol{\omega}}_{ob}^{b} &= \boldsymbol{J}^b \dot{\boldsymbol{\omega}}_{ib}^{b} - \boldsymbol{J}^b \left(\boldsymbol{q}_{ob}^* \otimes \dot{\boldsymbol{\omega}}_{io}^{o} \otimes \boldsymbol{q}_{ob} + \boldsymbol{\omega}_{io}^{b} \times \boldsymbol{\omega}_{ob}^{b}\right) \\
&= \boldsymbol{\tau}^b - \boldsymbol{\omega}_{ib}^{b} \times \boldsymbol{J}^b \boldsymbol{\omega}_{ib}^{b} - \boldsymbol{J}^b \left(\boldsymbol{q}_{ob}^* \otimes \dot{\boldsymbol{\omega}}_{io}^{o} \otimes \boldsymbol{q}_{ob} + \boldsymbol{\omega}_{io}^{b} \times \boldsymbol{\omega}_{ob}^{b}\right)
\end{aligned}$$

If we assume that we have a desired attitude relative to the orbit frame $\boldsymbol{q}_{od}, \boldsymbol{\omega}_{od}^{d}$ and $\dot{\boldsymbol{\omega}}_{od}^{d}$. Then we can design a $\boldsymbol{\tau}^b$ that will ensure as $t \to \infty$, $\left(\boldsymbol{q}_{ob}, \boldsymbol{\omega}_{ob}^{b}\right) \to \left(\boldsymbol{q}_{od}, \boldsymbol{\omega}_{od}^{d}\right)$. One such controller is

$$\begin{aligned}
\boldsymbol{\tau}^b = &-\left(1 + k_q k_\omega\right)\boldsymbol{q}_v - \left(k_\omega \boldsymbol{I} + \frac{k_q}{2}\boldsymbol{J}^b\left(q_0 \boldsymbol{I} + S\left(\boldsymbol{q}_v\right)\right)\right)\boldsymbol{\omega}_{db}^{b} + \boldsymbol{\omega}_{ib}^{b} \times \boldsymbol{J}^b \boldsymbol{\omega}_{ib}^{b} \\
&+ \boldsymbol{J}^b \left(\boldsymbol{q}_{ob}^* \otimes \dot{\boldsymbol{\omega}}_{io}^{o} \otimes \boldsymbol{q}_{ob} + \boldsymbol{\omega}_{io}^{b} \times \boldsymbol{\omega}_{ob}^{b}\right) + \boldsymbol{J}^b \left(\boldsymbol{q}_{db}^* \otimes \dot{\boldsymbol{\omega}}_{od}^{d} \otimes \boldsymbol{q}_{db} + \boldsymbol{\omega}_{od}^{b} \times \boldsymbol{\omega}_{db}^{b}\right)
\end{aligned}$$

where $\boldsymbol{q}_{db} = \boldsymbol{q}_{od}^* \otimes \boldsymbol{q}_{ob}$, $\boldsymbol{\omega}_{db}^{b} = \boldsymbol{\omega}_{ob}^{b} - \boldsymbol{q}_{db}^* \otimes \boldsymbol{\omega}_{od}^{d} \otimes \boldsymbol{q}_{db}$ and q_0, $\boldsymbol{q}_v$ are the real and vector part of $\boldsymbol{q}_{db}$. The controller might look complicated, but like most controllers it is essentially made up of 3 basic parts:

Feedback: $-\left(1 + k_q k_\omega\right)\boldsymbol{q}_v - \left(k_\omega \boldsymbol{I} + \frac{k_q}{2}\boldsymbol{J}^b\left(q_0 \boldsymbol{I} + S\left(\boldsymbol{q}_v\right)\right)\right)\boldsymbol{\omega}_{db}^{b}$

Cancellations: $\boldsymbol{\omega}_{ib}^{b} \times \boldsymbol{J}^b \boldsymbol{\omega}_{ib}^{b} + \boldsymbol{J}^b \left(\boldsymbol{q}_{ob}^* \otimes \dot{\boldsymbol{\omega}}_{io}^{o} \otimes \boldsymbol{q}_{ob} + \boldsymbol{\omega}_{io}^{b} \times \boldsymbol{\omega}_{ob}^{b}\right)$

[(xi)] An advantage of Reynold's algorithm is that we don't have to construct the rotation matrix, we just need to unit vectors.

Feedforward: $\boldsymbol{J}^b\left(\boldsymbol{q}_{db}^* \otimes \dot{\boldsymbol{\omega}}_{od}^d \otimes \boldsymbol{q}_{db} + \boldsymbol{\omega}_{od}^b \times \boldsymbol{\omega}_{db}^b\right)$

The feedforward terms appear because we are tracking a time-varying trajectory, if $\boldsymbol{q}_{od}$ is constant such that $\boldsymbol{\omega}_{od}^d = \dot{\boldsymbol{\omega}}_{od}^d = \boldsymbol{0}$ then the feedforward term is also zero. Often we can drop the cancellations and feedforward terms at the cost of degraded performance (this is assuming we know $\boldsymbol{J}^b$, which typically we don't) and if in addition we neglect $\frac{k_q}{2}\boldsymbol{J}^b\left(q_0\boldsymbol{I} + S\left(\boldsymbol{q}_v\right)\right)$[xii] then controller has the structure

$$\boldsymbol{\tau}^b = -k_1\boldsymbol{q}_v - k_2\boldsymbol{\omega}_{db}^b$$

which is just a PD controller.

Assignment 4.2

1. Extend the sat_lib.py with a Satellite class which has a rigid-body as a class member. With the following member functions

__init__: constructs a satellite with a given orientation, angular velocity and inertia matrix (used to construct the internal rigid-body)

update: Takes t, t_step as input, computes a control torque and calls update on the rigid-body.

get_state: returns the attitude and angular velocity.

2. Simulate a Satellite tracking a time-varying reference $\boldsymbol{q}_{od}, \boldsymbol{\omega}_{od}^d, \dot{\boldsymbol{\omega}}_{od}^d$ using a PD controller

$$\boldsymbol{\tau}^b = -k_1\boldsymbol{q}_v - k_2\boldsymbol{\omega}_{db}^b$$

Assume that $\mathcal{F}^o = \mathcal{F}^i$, i.e. $\boldsymbol{q}_{io} = \boldsymbol{q}_I$, $\boldsymbol{\omega}_{io}^o = \dot{\boldsymbol{\omega}}_{io}^o = \boldsymbol{0}$. Use the simulation parameters: t_0= 0, t_e= 500, t_step= 0.01, speed_factor = 1, anim_dt = 1/25, scale_factor= 1.

$$\begin{aligned}
\boldsymbol{q}_{ib}(0) &= \begin{bmatrix}1 & 0 & 0 & 0\end{bmatrix}^\top \\
\boldsymbol{\omega}_{ib}^b(0) &= \begin{bmatrix}0 & 0 & 0\end{bmatrix}^\top \\
\boldsymbol{J}^b &= \begin{bmatrix}0.5 & 0 & 0 \\ 0 & 0.5 & 0 \\ 0 & 0 & 0.5\end{bmatrix} \\
\boldsymbol{q}_{id}(0) &= \begin{bmatrix}\frac{1}{2} & \frac{1}{2} & \frac{1}{2} & \frac{1}{2}\end{bmatrix}^\top \\
\boldsymbol{\omega}_{id}^d(0) &= \begin{bmatrix}0.2 & -0.1 & 0.05\end{bmatrix}^\top \\
\dot{\boldsymbol{\omega}}_{id}^d &= \boldsymbol{0}
\end{aligned}$$

[xii] Observe that $\boldsymbol{q}_{db} \to \boldsymbol{q}_I \implies \frac{k_q}{2}\boldsymbol{J}^b\left(q_0\boldsymbol{I} + S\left(\boldsymbol{q}_v\right)\right) \to \frac{k_q}{2}\boldsymbol{J}^b$

A DCM to unit quaternion

A.1 Method 1: Shepperd's Algorithm

Probably the most well known algorithm is Shepperd's algorithm.

if $r_{11} + r_{22} + r_{33} > 0$

$$
\begin{aligned}
q_0 &= \pm\tfrac{1}{2}\sqrt{1 + r_{11} + r_{22} + r_{33}} \\
q_1 &= \tfrac{1}{4q_0}\left(r_{23} - r_{32}\right) \\
q_2 &= \tfrac{1}{4q_0}\left(r_{31} - r_{13}\right) \\
q_3 &= \tfrac{1}{4q_0}\left(r_{12} - r_{21}\right)
\end{aligned}
$$

else

$$
\begin{aligned}
&i = \text{row index of } \max(r_{11}, r_{22}, r_{33}) \\
&j = (i+1) \mod 4, \text{ and } k = (i+2) \mod 4 \\
&q_i = \pm\tfrac{1}{2}\sqrt{1 + r_{ii} - r_{jj} - r_{kk}} \\
&q_0 = \tfrac{1}{4q_i}\left(r_{jk} - r_{kj}\right) \\
&q_j = \tfrac{1}{4q_i}\left(r_{ij} + r_{ji}\right) \\
&q_k = \tfrac{1}{4q_i}\left(r_{ik} + r_{ki}\right)
\end{aligned}
$$

In this algorithm there is a sign ambiguity when computing q_0 or q_i and typically it is chosen such that $q_0 > 0$. A simple way of doing this is to first compute the quaternion using the positive square root and then at the end multiply the quaternion by the sign of q_0.

A.2 Method 2: Bar's Algorithm

In this algorithm given a matrix $\boldsymbol{R}$ (it is not required to be a rotation matrix), we first compute a 4 by 4 matrix $\boldsymbol{K}$

$$
\boldsymbol{K} = \frac{1}{3}\begin{bmatrix}
r_{11} - r_{22} - r_{33} & r_{12} + r_{21} & r_{13} + r_{31} & r_{32} - r_{23} \\
r_{12} + r_{21} & r_{22} - r_{11} - r_{33} & r_{23} + r_{32} & r_{13} - r_{31} \\
r_{13} + r_{31} & r_{23} + r_{32} & r_{33} - r_{11} - r_{22} & r_{21} - r_{12} \\
r_{32} - r_{23} & r_{13} - r_{31} & r_{21} - r_{12} & r_{11} + r_{22} + r_{33}
\end{bmatrix}
$$

Then we find the eigenvalues and eigenvector of $\boldsymbol{K}$

$$
\left(\lambda \boldsymbol{I} - \boldsymbol{K}\right)\boldsymbol{x} = \boldsymbol{0}
$$

Then we choose the eigenvector $\hat{\boldsymbol{x}} = \begin{bmatrix} x_0 & x_1 & x_2 & x_3 \end{bmatrix}^\top$ that has the corresponding eigenvalue with the largest real part (eigenvalues could be complex). Then

$$
\boldsymbol{q} = \begin{bmatrix} x_3 \\ x_0 \\ x_1 \\ x_2 \end{bmatrix}
$$

which is the unit quaternion that matches the coordinate transformation of $\boldsymbol{R}$ the closest. In the case when $\boldsymbol{R} \in SO(3)$, then $\boldsymbol{q}$ exactly matches. This feature is very useful for attitude determination, where $\boldsymbol{R}$ could be corrupted by noise. Like the previous algorithm we can multiply $\boldsymbol{q}$ by the sign of q_0 to always return a quaternion with $q_0 > 0$.

A.3 Method 3: Reynold's Algorithm

Denote the columns of the rotation matrix as

$$\boldsymbol{R} = \begin{bmatrix} a_1 & b_1 & c_1 \\ a_2 & b_2 & c_2 \\ a_3 & b_3 & c_2 \end{bmatrix} = \begin{bmatrix} \boldsymbol{a} & \boldsymbol{b} & \boldsymbol{c} \end{bmatrix}$$

and the vectors

$$\boldsymbol{e}_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \qquad \boldsymbol{e}_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \qquad \boldsymbol{e}_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

then the vector $\boldsymbol{x}$ can be computed in three equivalent ways

$$\boldsymbol{x} = \begin{bmatrix} b_1 - a_2 \\ (\boldsymbol{a} - \boldsymbol{e}_1) \times (\boldsymbol{b} - \boldsymbol{e}_2) \end{bmatrix} = \begin{bmatrix} c_1 - a_3 \\ (\boldsymbol{a} - \boldsymbol{e}_1) \times (\boldsymbol{c} - \boldsymbol{e}_3) \end{bmatrix} = \begin{bmatrix} c_2 - b_3 \\ (\boldsymbol{b} - \boldsymbol{e}_2) \times (\boldsymbol{c} - \boldsymbol{e}_3) \end{bmatrix}$$

The algorithm computes the magnitude of the three cross products and chooses the version which gave the largest value. Then $\boldsymbol{q}$ is found as

$$\boldsymbol{q} = \frac{1}{\|\boldsymbol{x}\|}\boldsymbol{x}$$

Just like the previous algorithms we can multiply $\boldsymbol{q}$ by the sign of q_0 to always return a quaternion with $q_0 > 0$.

B Example code

To illustrate how to handle projections and system with inputs we provide an example implementation of a class for a particle moving on a sphere. The system equations can be written as

$$\dot{\boldsymbol{p}}^i = \boldsymbol{v}^i$$
$$\dot{\boldsymbol{v}}^i = \frac{1}{m}\boldsymbol{f}^i - k \cdot \boldsymbol{v}^i$$

with constraints $\|\boldsymbol{p}^i\| = r$ and $\boldsymbol{p}^i \bullet \boldsymbol{v}^i = 0$. In Listing 1 we have implemented the equations of motion and as seen in line 16 the RK4 is oblivious to the input and the motion constraints. Since the RK4 does not respect the constraints of the system we take care of it ourselves in lines 19, 29 and 22-25. In addition the update function takes a force as input which is then stored in 'self.fs' in line 13. Then when 'f' is called by the RK4 algorithm we retrieve the force in line 36 and apply it to the system in line 38.

```
1  class ParticleOnSphere:
2    def __init__(self,p,v,m,k):
3      self.k = k
4      self.m = m
5      self.r = np.linalg.norm(p)
6      self.p = p
7      self.v = v
8      self.fs = np.array([0,0,0])
9
10   def update(self,t_k,h,fu):
11     #Set the forces acting on particle
12     fg = np.array([0,0,-self.m*g])
13     self.fs = fg+fu
14
15     x_k = np.concat((self.p,self.v))
16     x_k = su.step_RK4(t_k,x_k,h,self.f)
17
18     #Project position back to surface of sphere
19     self.p = self.r*x_k[:3]/np.linalg.norm(x_k[:3])
20
21     #Project velocity to be tangent to sphere
22     w = np.cross(np.cross(self.p,x_k[3:]),self.p)
23     if np.linalg.norm(w) > 1e-6:
24       w /= np.linalg.norm(w)
25     self.v = np.dot(x_k[3:],w)*w
26
27   def f(self,t,x):
28     #Project position back to surface of sphere
29     p = self.r*x[:3]/np.linalg.norm(x[:3])
30     dot_p = x[3:]
31
32     #Constrain the acting forces to be tangent to sphere.
33     w = np.cross(np.cross(p,self.fs),p)
34     if np.linalg.norm(w) > 1e-6:
35       w /= np.linalg.norm(w)
36     fc = np.dot(self.fs,w)*w
```

```

    dot_v = 1/self.m*fc-self.k*dot_p
    return np.concat((dot_p,dot_v))
```

Listing 1: Example implementation of a 3D inverted pendulum